

Technical Report: Real-time Bethe-Salpeter Equation (BSE) Implementation in paraHFNAMD

Gemini CLI

March 2, 2026

1 Introduction

The `paraHFNAMD` package implements a real-time Bethe-Salpeter Equation (BSE) solver designed to calculate exciton properties based on ground-state GW calculations. This report details the implementation of the BSE module, the functions involved, and their assembly into the overall workflow.

2 Core Classes and Data Structures

The BSE implementation is primarily housed in the `src/bse.cpp` and `src/bse.h` files, centered around the `excitonclass`. It interacts with several other key components:

- **waveclass**: Manages electronic wavefunctions, quasiparticle energies, and PAW (Projector Augmented Wave) information.
- **wpotclass**: Handles the screened Coulomb potential (W) obtained from a prior GW calculation.
- **socclass**: Handles spin-orbit coupling effects if requested.

3 BSE Hamiltonian Construction

The BSE Hamiltonian H^{BSE} is constructed in the basis of electron-hole pairs (ck, vk') . In the Tamm-Dancoff approximation, the matrix elements are:

$$H_{(ck,vk'),(c'k'',v'k''')} = (\epsilon_{ck} - \epsilon_{vk})\delta_{cc'}\delta_{vv'}\delta_{kk''}\delta_{k'k'''} + K_{(ck,vk'),(c'k'',v'k''')} \quad (1)$$

where $K = K^d + \kappa K^x$ is the interaction kernel.

4 Practical Implementation of $W_{GG'}(q)$

The screened Coulomb potential $W_{GG'}(q)$ is a central component of the BSE kernel. In `paraHFNAMD`, this is handled by the `wpotclass`.

4.1 Reading W from Files

The potential is typically read from binary files (e.g., `Wxxxx.tmp` or `WFULLxxxx.tmp`) generated by VASP's GW implementation.

- **Head and Wing**: The $q \rightarrow 0$ limits (W_{00} and W_{0G}) are read from `W0001.tmp` in `ReadHeadWing()`. The head represents the average of the diagonal dielectric tensor elements.
- **Full Matrix**: The `WFULLxxxx.tmp` files contain the full $W_{GG'}(q)$ matrices for each k-point in the irreducible Brillouin zone.
- **Memory-Efficient Reading**: To handle large matrices, `ReadOneWFullSub()` reads the matrix in blocks corresponding to irreducible k-points, minimizing the instantaneous memory footprint.

4.2 Reciprocal Space Mapping ($Q + G \rightarrow q + g$)

GW calculations often use a primitive cell, while BSE may use a supercell. The mapping is performed in `QpG2qpg()`:

1. A transformation matrix M is computed to relate supercell and primitive cell lattice vectors.
2. For each supercell $Q + G$ vector, the corresponding primitive cell $q + g$ vector is identified.
3. Symmetry operations (rotations, time-reversal) are applied using `Doublex012toqg()` to map any q -point back to the irreducible Brillouin zone where W is stored.

5 Practical Calculation of Interaction Kernels

5.1 Direct Interaction (K^d)

The calculation of K^d is implemented in `DirectTerms()`.

- **Data Distribution:** The calculation is parallelized over k -point pairs (k, k') using MPI. Each process handles a subset of pairs.
- **Matrix Operations:** For each pair, the code:
 1. Computes density matrices $M_{cc'}(k, q, G)$ and $M_{vv'}(k, q, G')$ using PAW-augmented wavefunctions.
 2. Performs the double contraction: $K^d = \alpha \times \text{trans}(M_{cc'}) \times W \times \text{conj}(M_{vv'})$.
 3. Uses high-performance BLAS `Zgemm` for the matrix multiplications.
- **Singularity Handling:** For $q = 0$, the head and wing contributions are added separately to handle the Coulomb singularity.

5.2 Exchange Interaction (K^x)

The exchange term is implemented in `ExchangeTerms()`.

- **Density Matrix:** It first computes the $q = 0$ density matrix $M_{cv}(k, 0, G)$ using `DensityMatrixKCV()`.
- **ScaLAPACK Integration:** Because the resulting BSE matrix is globally distributed in a 2D block-cyclic manner, the exchange term (which involves all excitonic transitions) is added using ScaLAPACK's `Pzgemm`.
- **Scaling:** Each term is scaled by the bare Coulomb potential $v(G) = 4\pi/|G|^2$ and the crystal volume Ω .

6 Numerical Diagonalization and Phase Alignment

After the Hamiltonian is assembled, `xEnergies()` performs the following:

1. **Diagonalization:** Uses `Pzheev` (ScaLAPACK) to find eigenvalues and eigenvectors.
2. **Phase Continuity:** In a real-time trajectory, the phase of eigenvectors can jump between steps. The code aligns the current eigenvectors $A(t)$ with the previous ones $A(t - dt)$ by calculating the overlap $\langle A(t - dt) | A(t) \rangle$ and applying a phase correction. This is essential for stable TDM and non-adiabatic coupling calculations.
3. **Exciton TDM:** Transition dipole moments are computed in `ExcitonTDM()` by weighting the momentum matrix elements $\mathbf{P}_{cvk}$ with the exciton coefficients A_{cvk}^S .

7 Summary of Functional Assembly

The `OnlyBSECalc()` driver coordinates the following sequence for each time step:

1. **Wavefunction Load:** `WCBasicProcess()` reads electronic states.
2. **Mapping:** `wpotclass` maps GW potential to the current basis.
3. **Kernel Construction:** `DirectTerms()` and `ExchangeTerms()` build the interaction matrices.
4. **Matrix Assembly:** `ScTerms2BSEMat()` and `ExTerms2BSEMat()` scatter local contributions into the global block-cyclic BSE Hamiltonian.
5. **Solution:** `XEnergies()` diagonalizes the Hamiltonian and extracts exciton properties.